Checked Generation for Lecture Decks: Computed Step-by-Step Explainers, Verified Interactive Labs and Sanitised Diagrams under Free-Tier Quotas

Swayam Devendra Panchal[1], Kawaljeet Singh Bharaj[1], Jeevesh Bharat Bodhani[1], Rishit Pankajkumar Lahoti[1], and Pratik Kanani[2]

[1] Student, Department of Artificial Intelligence and Data Science, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India
panchalswayam35@gmail.com
[2] Head of Department, Department of Artificial Intelligence and Data Science, Dwarkadas J. Sanghvi College of Engineering, Mumbai, India

Abstract. Learnova rebuilds the lecture decks that teachers already use into illustrated and interactive ones. Several of the things it adds—worked solutions, small laboratories, diagrams and rewritten titles—could be produced by a language model, but a model can make arithmetic errors, write unsafe code or drift from the slide. This paper describes the checks we designed so that every added item is either computed, verified or replaced by an offline result. We present (i) a detector and a library of exact solvers that turn a problem in slide text into a step-by-step explanation; (ii) a lab contract, a selection policy and a static gate for model-written lab code; (iii) a whitelist sanitiser for model-drawn SVG diagrams with label fitting; (iv) a lexical grounding filter for model-written titles and explanations; and (v) a paced key pool that keeps several free-tier keys busy and skips spent ones. On a held-out set of 26 problems the solvers gave the correct answer every time, with no false detections on 52 ordinary sentences. The sanitiser left no unsafe construct in 13 accepted diagrams out of 18 crafted answers, and the lab gate refused all 10 unsafe or broken programs after we fixed one gap the test exposed. With five keys the pool produced 40 diagrams in 42 s against 183 s with one key, within 15 s of a simple throughput bound, and remembering spent keys cut the time to reach the offline fallback from 166 s to 22 s.

Keywords: worked examples · step-by-step solutions · interactive labs · output validation · SVG sanitising · rate-limited scheduling · educational technology

1 Introduction

Most lecture decks in an engineering course are reused from year to year. They are dense, they show solved problems as finished tables, and they rarely contain anything a student can try. Learnova takes such a deck and rebuilds it: every

point of the source is kept, and each slide gets a suitable visual form, a title that states its point, an illustration and, where it helps, an explanation or a lab. The rebuilding itself is rule-based and described elsewhere. This paper is about the parts that a language model could write, and about how we make them safe to put in front of a student.

The risk is real. A model asked to solve a problem may produce a confident wrong step [10, 12]; a model asked for a diagram may return markup with scripts or links; a model asked for a lab returns code that will run in a student's browser; and a model asked to improve a title may write something unrelated to the slide [13]. In each case the cost of a bad output is higher than the cost of no output, because students trust what appears on a course slide.

We therefore follow one rule throughout: *a generated item is shown only if it passes a check that does not depend on the model, and when it fails, an offline result takes its place.* Where the item can be computed exactly, as with arithmetic and the dynamic programs taught in our courses, we do not ask the model at all.

A second constraint is cost. The college has no paid AI budget, so the system runs with free-tier keys, each limited to a few requests per minute and a daily quota. A deck may need forty diagrams and ten labs, so how requests are spread over the keys decides whether a deck takes one minute or five.

Our contributions are:

1. a problem detector with sixteen exact solvers that produce explained, step-by-step solutions (Sect. 4);
2. a lab contract, a lab selection policy and a static gate for model-written lab code, with an offline builder as the fallback (Sect. 5);
3. a whitelist sanitiser for model-drawn SVG and a label-fitting rule (Sect. 6);
4. a lexical grounding filter for model-written titles and explanations (Sect. 7);
5. a paced key pool with a throughput bound that sets the number of parallel workers (Sect. 8);
6. an evaluation of each check on held-out and adversarial inputs, including two faults the evaluation found (Sect. 9).

2 Related Work

Worked examples. Novices learn more from studying solved problems than from solving the same problems unaided, as long as each step is visible and its reason is stated [20, 1, 17]. Computer algebra services such as Wolfram|Alpha [23] show steps for mathematics, and intelligent tutoring systems give step-level feedback that approaches human tutoring [21], but both need the problem in a clean form and neither reads lecture slides.

Interactive visualisation. Meta-analyses of algorithm visualisation find that the learner's engagement, not the animation, predicts learning [9, 16]; PhET shows that inquiry-style simulations work at scale [22]. The ICAP framework [3] and the wider evidence for active learning [4] support moving students from

watching to doing, and retrieval practice [18] motivates the "predict, then reveal" prompts of our labs.

Generated slides and their faithfulness. Systems such as PPSGen [8], D2S [19] and DOC2PPT [5] generate new slides from documents, and commercial tools [6, 15, 7, 2] do the same from a prompt. None of them checks what it generates. Studies of abstractive summarisation show that unchecked models add unsupported content [13, 10], which is the problem our checks address. Table 1 places our work against these lines.

Table 1. Where generated content comes from and how it is checked

Approach	Source of steps or content	Check before display	Works without a key
Computer algebra services	symbolic engine	exact by design	no (web service)
Tutoring systems	hand-authored rules	exact, per domain	yes, per domain
Prompt-to-deck tools	language model	none	no
Document-to-slide models	trained model	none	yes, after training
Learnova (this work)	solvers, rules, model	computed, gated or sanitised	yes, offline fallback

3 Checked Generation

Figure 1 shows where the checks sit. For an input x (a slide) and a kind of item κ (explainer, lab, diagram, title), let $M_\kappa(x)$ be the model's answer, G_κ a check that does not use the model, and $F_\kappa(x)$ an offline result that may be empty. The item shown is

$$y_\kappa(x) = \begin{cases} S_\kappa(x) & \text{if an exact solver } S_\kappa \text{ applies,} \\ M_\kappa(x) & \text{else if a key is available and } G_\kappa(M_\kappa(x)) = 1, \\ F_\kappa(x) & \text{otherwise.} \end{cases} \tag{1}$$

The order matters. A computed answer is always preferred, because it is both correct and free. The model is used only where nothing can be computed, and only through a check. The offline path guarantees that a deck is always finished, even with no network at all.

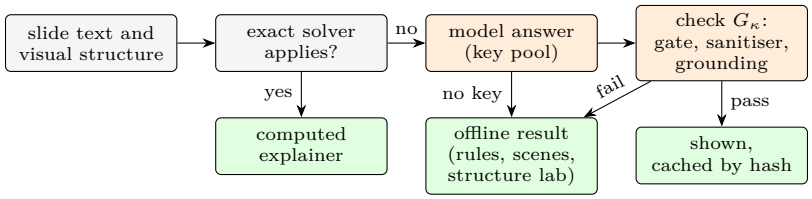


Fig. 1. Checked generation. Orange stages involve the model; green results are what the student sees.

4 Computed Step-by-Step Explainers

4.1 Representation

An explainer and each of its steps are tuples

$$E = (\kappa, p, \langle s_1, \ldots, s_n \rangle, a, \rho, \sigma), \qquad s_i = (\mathit{title}_i, \mathit{work}_i, \mathit{why}_i, \mathit{rule}_i, T_i) : \tag{2}$$

the problem type, the statement, the steps, the answer, a short recap and the source $\sigma \in \{\text{computed}, \text{slide}, \text{model}\}$. A step has a short action title, the lines of working with the changed part marked, a plain reason, the general rule, and for table algorithms the state of the table T_i with the cells to highlight. The page that shows an explainer prints its source, so a student can see whether a number was computed.

4.2 Detection

Slide text rarely holds a clean problem. A system of equations may sit on two lines, a grammar may fill the slide, and a problem may be one sentence in a paragraph. The detector builds candidates in a fixed order: the whole text, then each pair of neighbouring lines that both contain an equals sign, then each line, then each sentence:

$$\mathit{Cand}(x) = \langle x \rangle \parallel \langle \ell_i ; \ell_{i+1} : \text{“=”} \in \ell_i, \ell_{i+1} \rangle \parallel \langle \ell_i \rangle \parallel \langle \text{sentences of } \ell_i \rangle, \tag{3}$$

with duplicates and strings longer than 4000 characters removed. Each candidate is offered to the solvers $\mathcal{S} = \langle S_1, \ldots, S_{16} \rangle$ in a fixed order, and the first explainer with at least three steps wins:

$$E^*(x) = \text{first}\{\, S_j(c) : c \in \mathit{Cand}(x),\ j = 1..16,\ |S_j(c)| \geq 3 \,\}. \tag{4}$$

The solver order puts the most specific patterns first (Viterbi, Naive Bayes, bigram, CYK, Bayes' rule) and the most general last (percentages, statistics, speed), so a Bayes problem that contains a percentage is not taken by the percentage solver.

For algebra the equation is cut out of the sentence by removing leading words ("Solve", "Find x if", "Solve for y given") and trailing clauses. The result is then rejected if either side still contains a word:

$$\mathit{reject}(q) \iff \exists\, w \in \mathit{lhs}(q) \cup \mathit{rhs}(q) : w \text{ matches } [A\text{–}Za\text{–}z]\{2,\}. \tag{5}$$

This rule is deliberately strict. A sentence such as "Version 2 of the system was released in 2026" contains numbers but no equation, and it is the strictness of Eq. 5 that keeps such sentences from being treated as problems.

Algorithm 1 Finding and solving a problem on a slide

Require: slide text x, optional worked-example rows W found by the visual engine
for $c \in \mathit{Cand}(x)$ **do**
  for $S_j \in \mathcal{S}$ in order **do**
    $E \leftarrow S_j(c)$ ▷ exceptions count as no answer
    if $E \neq \varnothing$ and $|E| \geq 3$ **then return** E with source "computed"
    end if
  end for
end for
if W has ≥ 2 rows and $\geq 60\%$ of rows have a reason or a known verb **then**
  return E built from W with source "slide"
end if
if a key is available and x looks like a problem **then**
  return model explainer if it passes Eq. 12
end if
return $\varnothing$

4.3 Solvers

All arithmetic uses SymPy [14] or exact fractions, so an answer such as 3/28 is shown as a fraction and as a decimal and is never rounded in the middle of a calculation. Table 2 lists the solvers and what their steps show.

Table 2. The sixteen solvers and the content of their steps

Solver	Steps
Linear equation	write, expand, collect the unknown, remove the constant, divide, check by substitution
Quadratic equation	standard form, factor or formula, both roots, check
Pair of linear equations	elimination with the multiplier shown, back-substitution, check
Percentage, simple interest, speed	formula, substitution, result with units
Mean, median	sort or sum, count, divide or pick the middle
Probability, Bayes' rule	name the quantities, numerator, denominator, result
Pythagoras	formula, squares, sum, square root
Derivative, integral	rule applied term by term, then combined
CYK parsing	the chart cell by cell, with the split points and the rules that fire
Bigram probability	counts, each conditional probability, product
Naive Bayes classification	priors, smoothed likelihoods, scores, decision
Viterbi decoding	initialisation, recursion per time step, back-pointers, path

Four solvers implement the dynamic programs of our course and are the reason we built exact solvers at all. For a bigram model with sentence markers, the maximum-likelihood estimate [11] is computed for every pair of the query sentence:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1} w_i)}{C(w_{i-1})}, \qquad P(w_{1:n}) = \prod_{i=1}^{n} P(w_i \mid w_{i-1}). \tag{6}$$

For Naive Bayes with classes s, vocabulary size V and N_s word tokens in class s, the solver shows the add-one estimate and the decision:

$$\hat{s} = \arg\max_{s} \; P(s) \prod_{j} P(f_j \mid s), \qquad P(f_j \mid s) = \frac{C(f_j, s) + 1}{N_s + V}. \tag{7}$$

For an HMM with start probabilities π_j, transitions a_{ij} and emissions $b_j(o)$, the Viterbi solver writes every candidate product, not just the maximum, because the comparison is the part students get wrong:

$$v_1(j) = \pi_j \, b_j(o_1), \qquad v_t(j) = \max_{i} \; v_{t-1}(i) \, a_{ij} \, b_j(o_t), \tag{8}$$

$$\psi_t(j) = \arg\max_{i} \; v_{t-1}(i) \, a_{ij}. \tag{9}$$

For CYK with a grammar in Chomsky normal form, one step is one cell of the chart:

$$T[i, i] = \{A : A \to w_i\}, \tag{10}$$

$$T[i, j] = \bigcup_{k=i}^{j-1} \{A : A \to B\,C, \; B \in T[i, k], \; C \in T[k{+}1, j]\}. \tag{11}$$

4.4 Model fallback

A problem that no solver recognises may be explained by the model when a key is available. Such an explainer is marked "model" on the page, and it is accepted only if the text looked like a problem in the first place and the answer has a usable shape:

$$G_{\text{expl}}(x, E) = [\,\pi(x)\,] \cdot [\,3 \le |E| \le 8\,] \cdot [\,a \neq \varnothing\,] \cdot \prod_i [\,\mathit{title}_i \neq \varnothing\,], \qquad (12)$$

where $\pi(x)$ is true when x contains a digit or a problem verb such as *solve*, *compute*, *parse* or *estimate*. The gate is weaker than a computed answer, which is why the page says where the steps came from.

5 Interactive Labs

5.1 The lab contract

All labs share one page: presets, a view of the current state, a "what is happening" panel with Step, Back and Play controls, and a "try your own input" form. A lab supplies only its presets, two form functions and one function that turns an input into a list of frames:

$$\text{build}(u) = \langle f_1, \ldots, f_m \rangle, \qquad f_k = (\mathit{label}_k, \mathit{title}_k, \mathit{explain}_k, \mathit{view}_k), \qquad (13)$$

where each frame is one small decision with its explanation and the view to draw. The last frame must be labelled *Answer*. Because the contract is this small, a hand-written lab is one function, and the same contract is what we give to the model when it writes a lab.

5.2 Choosing labs for a deck

Each built-in lab declares topic phrases, and a slide gets that lab when its title or text matches one. Other slides may get a generated lab, subject to a policy that keeps the deck usable (Table 3). Writing $\lambda(i)$ for the lab of slide i and $\tau(i)$ for its topic, the policy is

$$\lambda(i) = \lambda(j) \text{ if } \tau(i) = \tau(j), \quad |\{i : \lambda(i) = \ell\}| \le 2\ \forall \ell, \quad |\{\ell \text{ generated}\}| \le 10. \qquad (14)$$

Table 3. Lab selection policy

Rule	Reason
One lab per topic	continuation slides of one topic share their lab
Same lab at most twice per deck	a lab repeated on every slide stops being noticed
At most ten generated labs	bounds build time and key use
No lab on agenda, summary or quiz slides	these slides have nothing to step through
No lab for debris-like titles	fewer than 70% letters, unbalanced brackets, or no word of four letters
Offline lab needs $\geq$ 3 structured items	a two-item walk-through teaches nothing

5.3 The gate for model-written labs

With a key, the model receives the topic and the slide text and returns JSON with a title, form fields and JavaScript. The code will run in a student's browser, so it passes a static gate before it is used (Algorithm 2). A lab must not reach the network or storage, must not evaluate strings as code, must not break out of its script element, must define the four names of the contract, and must parse.

Algorithm 2 Gate for model-written lab code

Require: model reply r
1: $o \leftarrow$ first JSON object in r; **if** none, or `js`, `inputs`, `title` are not strings **return** reject
2: $j \leftarrow o$.`js`
3: **if** j contains `</script>` (any case) **then return** reject
4: **end if**
5: **if** j uses `fetch`, `XMLHttpRequest`, `eval`, `document.cookie`, **then**
6: `localStorage` or `import(` as a whole word **return** reject
7: **end if**
8: **if** any of `PRESETS`, `function build`, `function fill`, `function readInputs` is missing **then return** reject
9: **end if**
10: **if** `node -check` fails on stubs $+ j$ **then return** reject
11: **end if**
12: **return** accept

5.4 Offline labs

Without a key, or when the gate rejects the model's lab, the lab is built from the visual structure that the decision engine found on the slide. Table 4 shows how slide families map to the four kinds of offline walk-through. The student

can edit the points and replay, so even the offline lab is an exercise rather than a slideshow.

Table 4. Offline lab kinds built from the slide's own structure

Slide family	Lab kind	What one frame shows
process, cycle, cause–effect, layers, funnel, worked example	process	one step, linked to the step before and after
timeline	timeline	one event in order
comparison table or visual	compare	one criterion across all options
key points, list, outline, definition	concepts	"think first" prompt, then the slide's explanation

Labs are generated in parallel before the deck is built and cached by a hash of their content, so rebuilding a deck costs nothing.

6 Sanitised Diagrams

Each topic slide may get a diagram drawn by the model as SVG. SVG is a document format that can carry scripts, event handlers, embedded HTML, remote images and links, so we never insert the model's markup directly. An early version rejected any answer that contained a disallowed element; it accepted only 71 of 206 diagrams (34.5%), mostly because of harmless extras such as a `style` block. The current version sanitises instead of rejecting. It parses the answer as XML, keeps only whitelisted elements (shapes, text, groups, gradients, markers, clip paths and `use`), turns inline style declarations into attributes, deletes event-handler attributes and `url()` references that leave the document, and rejects the diagram only when something cannot be repaired. The acceptance condition is

$$G_{\mathrm{svg}}(d) = [\,\text{parses}\,] \cdot [\,\text{no external href}\,] \cdot [\,n_{\mathrm{shape}} \geq 5\,] \cdot [\,n_{\mathrm{text}} \leq 28\,] \cdot [\,|\mathrm{PNG}(d)| \geq 400 \text{ bytes}\,], \quad (15)$$

where the last factor rejects diagrams that render as an empty image. An external `href` is treated as unrepairable, because removing it from a `use` element changes what the diagram shows.

Labels are then fitted. With font size s, label length n characters, anchor position x and canvas width W_v taken from the diagram's own `viewBox`, the estimated width and the room available are

$$w = 0.56\, s\, n, \qquad R = \begin{cases} 2\min(x - 12,\, W_v - 12 - x) & \text{middle anchor} \\ x - 12 & \text{end anchor} \\ W_v - 12 - x & \text{start anchor} \end{cases}, \quad (16)$$

$$s' = \max\left(11,\, s\,\frac{R}{w}\right) \quad \text{if } w > R > 0. \quad (17)$$

The accepted diagram is stored, rendered to PNG for the PowerPoint file, and served as an HTML page that the picture on the slide links to.

7 Grounding Filter for Titles and Explanations

The model also proposes assertion titles (a full sentence that states the slide's point) and, for thin slides, a short explanation. These are checked against the slide's own words. Let $C(t)$ be the set of words of at least four letters in t, minus a short stop list. For a slide x, a proposed title h and explanation e:

$$G_{\text{title}}(x,h) = [\,4 \le |h|_w \le 15\,] \cdot [\,|h|_c \le 100\,] \cdot [\,|C(h) \cap C(x)| \ge 1\,] \cdot [\,\text{first}(h) \notin \mathcal{I}\,], \tag{18}$$

$$G_{\text{expl}}(x,e) = [\,|e|_w \le 100\,] \cdot [\,|C(e) \cap C(x)| \ge 2\,], \tag{19}$$

where $|\cdot|_w$ and $|\cdot|_c$ count words and characters and $\mathcal{I}$ is a list of 36 imperative verbs (*compare*, *identify*, *explain*, ...) that would turn the title into an instruction. A title that fails keeps the source heading; an explanation that fails is dropped. The filter is cheap and lexical, and it is not a fact checker: it catches an answer about the wrong topic, not a subtle error on the right one.

8 The Paced Key Pool

8.1 Pacing and throughput

Each free-tier key allows about 15 requests per minute. We space calls on each key by $\tau = 4.2\,\text{s}$ with a per-key lock, so request r on key k starts at

$$t_r = \max\big(t_{\text{ready}},\ t_{\text{last}}(k) + \tau\big). \tag{20}$$

With k live keys and w worker threads, each request holds a worker for at least the model latency L. The two limits give a lower bound on the time for N requests:

$$T(N,k,w) \ \ge\ N \cdot \max\!\left(\frac{\tau}{k},\ \frac{L}{w}\right). \tag{21}$$

The first term is the pace limit and the second the worker limit. Neither should dominate, so we want $L/w \le \tau/k$, that is $w \ge k\,L/\tau$. For diagram requests L/τ is about two, which is why the pool runs two requests in flight per key:

$$w = \max\big(4,\ \min(12,\ 2k)\big). \tag{22}$$

Requests are spread round-robin: request r tries the keys starting from key number $r \bmod K$.

8.2 Spent keys

A key that returns an authorisation error or reports its daily quota used up is recorded with the time $d(k)$, and it is not tried again until the retry interval $\Delta = 900\,\mathrm{s}$ has passed:

$$\mathit{live}(t) = \{\, k : t - d(k) > \Delta \,\}. \tag{23}$$

When $\mathit{live}(t)$ is empty the request goes straight to the router's other providers, and if they are unavailable too, to the offline path of Eq. 1. Algorithm 3 summarises one request.

Algorithm 3 One diagram request through the key pool

Require: prompt p, keys K, counter r
1: $O \leftarrow$ rotation of K starting at $r \bmod |K|$; $\quad r \leftarrow r + 1$
2: **for** $k \in O$ with $t - d(k) > \Delta$ **do**
3: wait until $t \geq t_{\mathrm{last}}(k) + \tau$; $\quad t_{\mathrm{last}}(k) \leftarrow t$
4: **try return** generate(k, p)
5: **on error** e: **if** e is an authorisation or quota error **then** $d(k) \leftarrow t$
6: **end for**
7: **return** router fallback (Groq, then NVIDIA), or $\varnothing$ for the offline path

9 Evaluation

Every number in this section is produced by two scripts in the project repository: one for the explainer benchmark and one for the remaining experiments. The key-pool runs use the real scheduler with the model call replaced by a fixed delay, scaled ten times faster and reported in real seconds.

9.1 Explainers

We wrote 26 problems that do not appear in our unit tests, with new numbers and new wordings across all solver types, and worked out each answer by hand. We also wrote 52 ordinary sentences that must not be treated as problems: 12 from lecture notes and 40 about the system itself, many containing numbers, units, years or equation-like words. While checking, two of our own hand answers turned out to be wrong (a bigram product and a Viterbi path); the solvers were right in both cases.

Table 5. Held-out explainer benchmark

Measure	Result
Problems detected (first run)	25 / 26
Problems detected (after one parsing fix)	26 / 26
Correct problem type	26 / 26
Correct answer	26 / 26
Every step has a reason	26 / 26
Mean steps per explainer	5.1
Median solve time	0.7 ms
False detections on ordinary sentences	0 / 52

The single miss in the first run was "Find x if $4x+9 = x+21$": the extractor did not remove the words before the equation. We added the pattern and two regression tests, and we report both numbers because the fix was made after seeing the benchmark. Figure 2 shows how long the explanations are. CYK produces the longest, one step per chart cell, while percentages and speed need three steps.

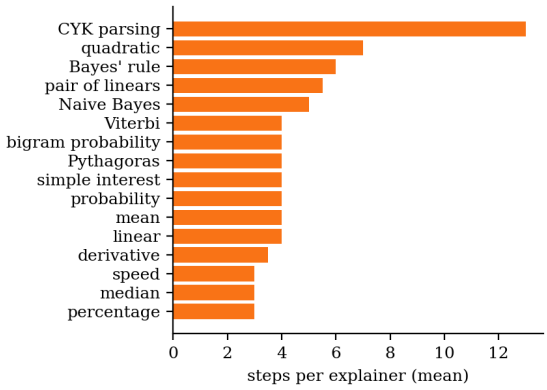


Fig. 2. Mean number of steps per explainer, by problem type, on the held-out benchmark.

9.2 Diagram sanitiser

We wrote 18 model-style answers: four ordinary diagrams (one wrapped in prose and a code fence, one with gradients and markers, one with inline styles), nine with a hostile or unsupported part, and five that are broken or out of bounds. Table 6 gives the outcome. The output of every accepted diagram was searched for scripts, event handlers, `foreignObject`, `javascript:` URLs, remote images and external links.

Table 6. Sanitiser on 18 crafted answers

Answer contains	Outcome	Unsafe content left
plain diagram; prose and code fence; gradients and markers; inline styles	accepted (4/4)	none
script element; event handler; `foreignObject`; remote image; link element; remote `url()`; animation; style sheet	accepted after cleaning (8/8)	none
external `href` on `use`	rejected	—
two shapes only; 35 labels	rejected (2/2)	—
truncated answer; malformed XML	rejected (2/2)	—
over-long label at the right edge	accepted, font 22 → 11	none
Total	13 accepted, 5 rejected	0 of 13

The test exposed one fault. Label fitting used the width of our slide panel (600 units) instead of the diagram's own `viewBox`, so a label at $x = 640$ on a 720-wide canvas had negative room and was left at 22 pt. Taking W_v from the `viewBox`, as in Eq. 16, fixed it.

9.3 Lab gate

Table 7 lists twelve model replies, from a correct lab to replies that try the network, storage or string evaluation. The first run accepted the dynamic `import()` case. The pattern had a word boundary after the opening bracket, which can never match; we moved `import(` to its own pattern and added a regression test. The table shows the result after the fix. Pattern checks take well under a millisecond; the `node -check` step dominates the time of an accepted lab.

Table 7. Lab gate on twelve model replies (after the fix)

Reply	Outcome	Time
well-formed lab	accepted	387 ms
calls `fetch`; uses `XMLHttpRequest`	rejected	< 1 ms
dynamic `import()`	rejected (accepted before fix)	< 1 ms
uses `eval`; reads cookies; writes local storage	rejected	< 1 ms
closes its script tag	rejected	< 1 ms
no `build`; no `PRESETS`	rejected	< 1 ms
syntax error	rejected by Node	57 ms
reply is not JSON	rejected	< 1 ms

9.4 Grounding filter

We wrote three slides about the system and seven model answers for them, each designed to test one condition of the filter. Table 8 shows the outcome.

Table 8. Grounding filter on seven crafted answers

Answer	Title	Explanation
on-topic assertion title, on-topic explanation (22 words)	kept	kept
title and explanation about an unrelated subject	dropped	dropped
title that is an instruction (“Compare . . . ”)	dropped	—
one-word topic label as title	dropped	—
on-topic title of 24 words	dropped	—
on-topic title, repetitive explanation of 95 words	kept	kept
on-topic title, good explanation (20 words)	kept	kept

The sixth row is a limitation: a repetitive explanation that stays on topic and under 100 words passes. The filter keeps the deck on topic; it does not judge writing quality.

9.5 Key pool

We sent 40 diagram requests through the real scheduler with one to five keys, a pace of $\tau = 4.2\,\mathrm{s}$ and a fixed latency of $L = 9\,\mathrm{s}$. Figure 3 and Table 9 compare the measured time with Eq. 21. The measured time stays within 15 s of the bound at every k; the gap is mostly the last request’s latency, which the bound leaves out. Five keys are 4.3 times faster than one.

Table 9. Time for 40 diagram requests against the number of keys

Keys k	Workers w	Bound (s)	Measured (s)	Gap (s)	Speed-up
1	4	168	183	15	1.0×
2	4	90	98	8	1.9×
3	6	60	70	10	2.6×
4	8	45	52	7	3.5×
5	10	36	42	6	4.3×

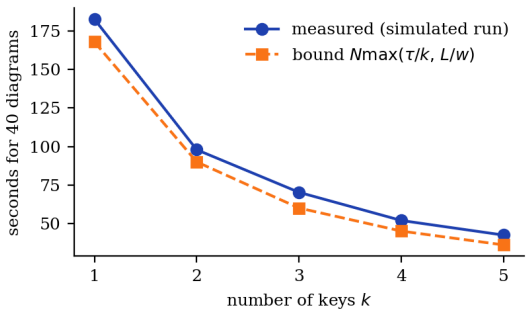


Fig. 3. Measured time for 40 diagrams against the bound of Eq. 21.

We then made some keys fail with a daily-quota error (Table 10). Before this study the diagram path did not remember spent keys, so every request tried each spent key again and paid a pacing slot for it. With some keys still alive this cost little, because the live keys set the pace. With every key spent, however, each request waited through all the dead keys before falling back, and the deck spent 166 s getting nothing. Remembering spent keys for Δ brought that to 22 s.

Table 10. Failover with spent keys (40 requests)

Keys	Spent	Served	Before (s)	After (s)
5	2	40	73	73
5	4	40	175	175
3	3	0 (offline path)	166	22

9.6 Whole decks

Figure 4 shows the build time of a 36-page deck in four modes. Offline rules alone take about 10 s. With every key dead the deck still finishes in under 30 s, because the router switches failing providers off and the offline paths take over. With working keys, generating labs one after another took 163 s; generating them in parallel before the build brought it to 74 s. On three course decks of 36, 95 and 216 pages built without a key, the policy of Table 3 attached labs to 6, 11 and 14 distinct topics; the largest deck reached the cap of ten generated labs.

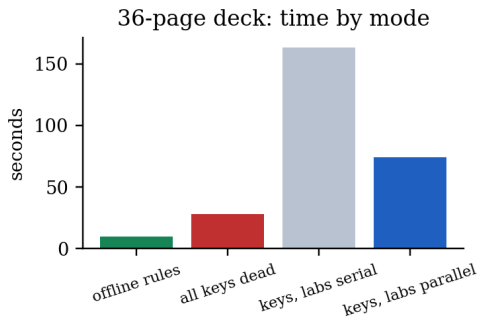


Fig. 4. Build time of a 36-page deck by mode.

10 Discussion and Limitations

Exact solvers cover a closed set of problem types, and that is a deliberate trade. Inside the set the explanation is always right and takes under a millisecond; outside it the page says honestly that the steps came from a model, or shows nothing. We would rather add solvers one at a time than widen the model's role.

Two of the checks found real faults in our own code: the dynamic-import gap in the lab gate and the canvas width in label fitting. A third experiment showed that a recovery path we thought was fine wasted almost three minutes when every key was spent. None of these appeared in normal use, which is an argument for testing a checker with inputs written to break it.

The checks have limits. The grounding filter is lexical and passes a repetitive but on-topic explanation. The lab gate is static; a lab that passes it can still show a wrong intermediate value, although it cannot reach the network. The key-pool results use a fixed latency, and real latency varies. Finally, we have measured correctness, coverage and time, not learning. Whether these explainers and labs improve examination results needs a classroom study, which we plan for the next semester.

11 Conclusion

A lecture deck can gain worked solutions, labs, diagrams and better titles without trusting a language model blindly. Computing what can be computed, checking what the model writes with rules that do not depend on the model, and keeping an offline path for everything else gave us explanations that were right on every held-out problem, labs and diagrams with no unsafe content left, and a deck that finishes even when every key is spent. A simple throughput bound told us how many requests to keep in flight, and testing the checks with hostile inputs found faults that ordinary use had not.

Acknowledgments. We thank the Department of Artificial Intelligence and Data Science, DJSCE, for guidance and for the course material used in this study.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Atkinson, R.K., Derry, S.J., Renkl, A., Wortham, D.: Learning from examples: Instructional principles from the worked examples research. Review of Educational Research **70**(2), 181–214 (2000)
2. Canva: Magic design for presentations. `https://www.canva.com/magic-design/` (2026), accessed Oct. 2026
3. Chi, M.T.H., Wylie, R.: The ICAP framework: Linking cognitive engagement to active learning outcomes. Educational Psychologist **49**(4), 219–243 (2014)
4. Freeman, S., Eddy, S.L., McDonough, M., Smith, M.K., Okoroafor, N., Jordt, H., Wenderoth, M.P.: Active learning increases student performance in science, engineering, and mathematics. Proceedings of the National Academy of Sciences **111**(23), 8410–8415 (2014)
5. Fu, T.J., Wang, W.Y., McDuff, D., Song, Y.: DOC2PPT: Automatic presentation slides generation from scientific documents. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 36, pp. 634–642 (2022)
6. Gamma Tech: Gamma: AI-powered presentations. `https://gamma.app` (2026), accessed Oct. 2026
7. Google: Gemini in Google Slides. `https://workspace.google.com/products/slides/` (2026), accessed Oct. 2026
8. Hu, Y., Wan, X.: PPSGen: Learning-based presentation slides generation for academic papers. IEEE Transactions on Knowledge and Data Engineering **27**(4), 1085–1097 (2015)
9. Hundhausen, C.D., Douglas, S.A., Stasko, J.T.: A meta-study of algorithm visualization effectiveness. Journal of Visual Languages and Computing **13**(3), 259–290 (2002)
10. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. ACM Computing Surveys **55**(12), 1–38 (2023)
11. Jurafsky, D., Martin, J.H.: Speech and Language Processing. Stanford University, 3rd (draft) edn. (2025), online manuscript, `https://web.stanford.edu/~jurafsky/slp3/`
12. Kasneci, E., et al.: ChatGPT for good? On opportunities and challenges of large language models for education. Learning and Individual Differences **103**, 102274 (2023)
13. Maynez, J., Narayan, S., Bohnet, B., McDonald, R.: On faithfulness and factuality in abstractive summarization. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. pp. 1906–1919 (2020)
14. Meurer, A., Smith, C.P., Paprocki, M., Čertík, O., et al.: SymPy: Symbolic computing in Python. PeerJ Computer Science **3**, e103 (2017)
15. Microsoft: Copilot in PowerPoint. `https://support.microsoft.com/copilot-powerpoint` (2026), accessed Oct. 2026

16. Naps, T.L., Rößling, G., Almstrum, V., Dann, W., Fleischer, R., Hundhausen, C., Korhonen, A., Malmi, L., McNally, M., Rodger, S., Velázquez-Iturbide, J.Á.: Exploring the role of visualization and engagement in computer science education. ACM SIGCSE Bulletin **35**(2), 131–152 (2003)
17. Renkl, A.: Toward an instructionally oriented theory of example-based learning. Cognitive Science **38**(1), 1–37 (2014)
18. Roediger, H.L., Karpicke, J.D.: Test-enhanced learning: Taking memory tests improves long-term retention. Psychological Science **17**(3), 249–255 (2006)
19. Sun, E., Hou, Y., Wang, D., Zhang, Y., Wang, N.X.R.: D2S: Document-to-slide generation via query-based text summarization. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics. pp. 1405–1418 (2021)
20. Sweller, J., Cooper, G.A.: The use of worked examples as a substitute for problem solving in learning algebra. Cognition and Instruction **2**(1), 59–89 (1985)
21. VanLehn, K.: The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. Educational Psychologist **46**(4), 197–221 (2011)
22. Wieman, C.E., Adams, W.K., Perkins, K.K.: PhET: Simulations that enhance learning. Science **322**(5902), 682–683 (2008)
23. Wolfram Research: Wolfram|Alpha step-by-step solutions. `https://www.wolframalpha.com/pro/step-by-step-math-solver` (2026), accessed Oct. 2026